

Artificial Neural Networks

Hands-on practices with sklearn package

scikit-learn
Machine Learning in Python

[Getting Started](#)

[Release Highlights for 0.24](#)

[GitHub](#)

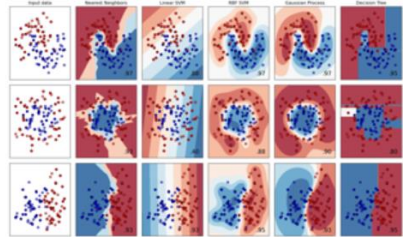
- Simple and efficient tools for predictive data analysis
- Accessible to everybody, and reusable in various contexts
- Built on NumPy, SciPy, and matplotlib
- Open source, commercially usable - BSD license

Classification

Identifying which category an object belongs to.

Applications: Spam detection, image recognition.

Algorithms: SVM, nearest neighbors, random forest, and more...



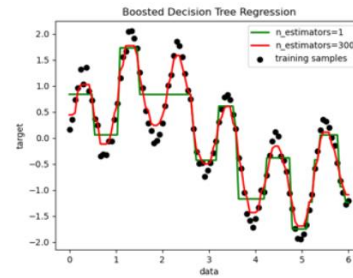
Examples

Regression

Predicting a continuous-valued attribute associated with an object.

Applications: Drug response, Stock prices.

Algorithms: SVR, nearest neighbors, random forest, and more...



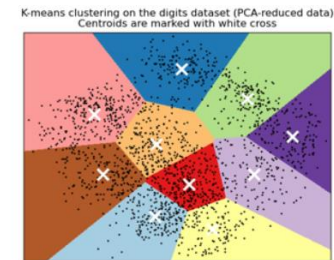
Examples

Clustering

Automatic grouping of similar objects into sets.

Applications: Customer segmentation, Grouping experiment outcomes

Algorithms: k-Means, spectral clustering, mean-shift, and more...



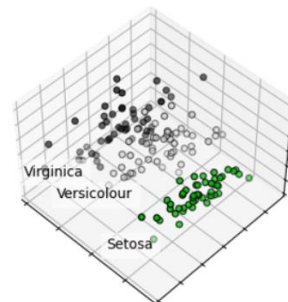
Examples

Dimensionality reduction

Reducing the number of random variables to consider.

Applications: Visualization, Increased efficiency

Algorithms: k-Means, feature selection, non-negative matrix factorization, and more...



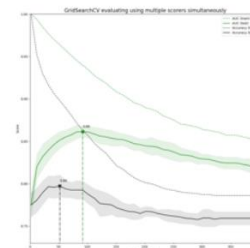
Examples

Model selection

Comparing, validating and choosing parameters and models.

Applications: Improved accuracy via parameter tuning

Algorithms: grid search, cross validation, metrics, and more...



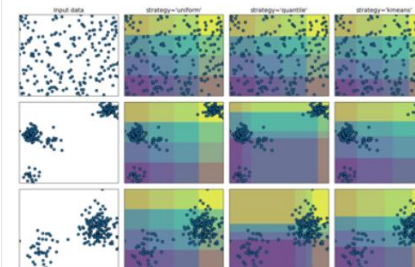
Examples

Preprocessing

Feature extraction and normalization.

Applications: Transforming input data such as text for use with machine learning algorithms.

Algorithms: preprocessing, feature extraction, and more...



Examples

Diabetes Detection

The dataset contains 768 observations and 8 variables, as described below:

1. pregnancies - Number of times pregnant.
2. glucose - Plasma glucose concentration.
3. diastolic - Diastolic blood pressure (mm Hg).
4. triceps - Skinfold thickness (mm).
5. insulin - Hour serum insulin (mu U/ml).
6. bmi – Basal metabolic rate (weight in kg/height in m).
7. dpf - Diabetes pedigree function.
8. age - Age in years.

Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	ClassLabel
6	148	72	35	0	33.6	0.627	50	1
1	85	66	29	0	26.6	0.351	31	0
8	183	64	0	0	23.3	0.672	32	1
1	89	66	23	94	28.1	0.167	21	0
0	137	40	35	168	43.1	2.288	33	1

Steps to build a neural network for diabetes detection:

1. Load dataset
2. Split data for training and testing
3. Normalize the feature values
4. Build neural network model
5. Train the neural network
6. Evaluate the trained neural network

```
1  # Step 1. Load the data
2  import numpy
3
4  # read data from a text file
5  data = numpy.loadtxt('diabetes.txt', delimiter = '\t')
6
7  X = data[:, :-1]      # expected input
8  y = data[:, -1]      # expected output
9
10 # view sample data
11 print(X[:5, :])
12 print(y[:5])
```

```
1 # Step 2. Split data for training and testing
2 from sklearn.model_selection import train_test_split
3
4 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=0)
5
6 # show the number of train and test samples
7 print('# of training samples:', len(y_train))
8 print('# of test samples:', len(y_test))
```

```
1 # Step 3. Normalize the feature values
2 from sklearn.preprocessing import StandardScaler
3
4 scaler = StandardScaler()
5 scaler.fit(X_train)
6 X_train = scaler.transform(X_train)
7 X_test = scaler.transform(X_test)
8
9 # view data after normalization
10 print(X_train[:5, :])
11 print(y_train[:5])
```



```
1  # Step 4. Build neural network model
2  from sklearn.neural_network import MLPClassifier
3
4  # Size of input layer = 8
5  # Number of hidden layers = 1
6  # Size of hidden layer = 2
7  # Size of output layer = 1
8  model = MLPClassifier(hidden_layer_sizes=(2),
9                          activation='relu',
10                         learning_rate_init=0.001,
11                         max_iter=2000,
12                         random_state=2,
13                         verbose=True)
```



```
1  # Step 5. Train the designed model
2  model.fit(X_train, y_train)
3
4  # Show the weights and biases of the trained network
5  print("Weights = ", model.coefs_ )
6  print("Biases = ", model.intercepts_)
7  print("Size of output layer: ", model.n_outputs_ )
8  print("Activation function of output layer: ", model.out_activation_)
9
10 # Show the plot of the loss curve
11 import matplotlib.pyplot as plt
12 loss_values = model.loss_curve_
13 plt.plot(loss_values)
14 plt.xlabel("Iteration")
15 plt.ylabel("Cost value")
16 plt.show()
```

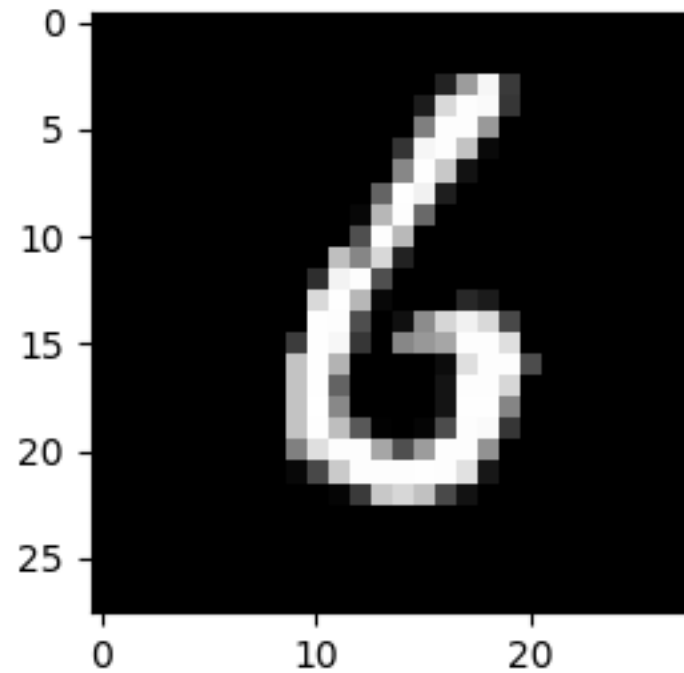
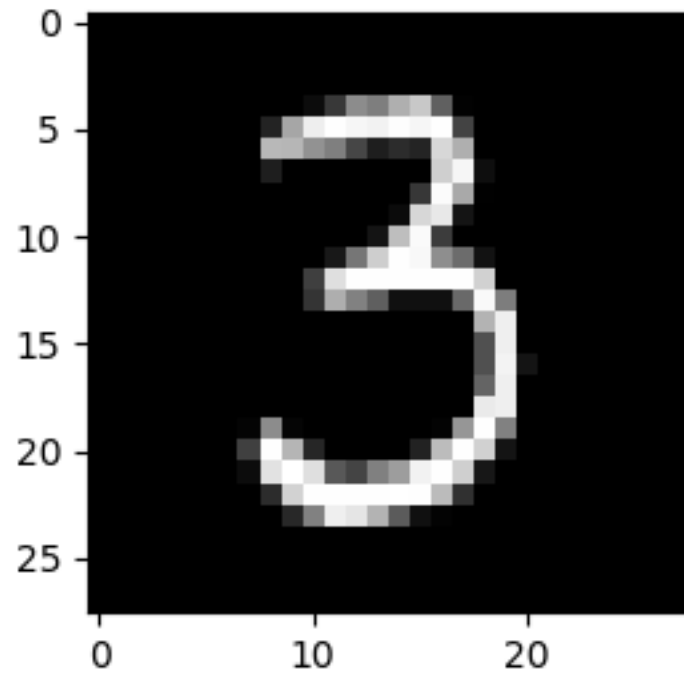
```
1 # Step 6. Evaluate the trained neural network
2 from sklearn.metrics import accuracy_score
3
4 # compute the accuracy of the trained network on the training set
5 predict_train = model.predict(X_train)
6 accuracy_on_training_set = accuracy_score(y_train, predict_train)
7 print("Accuracy on the training set: %.2f %" % (100 * accuracy_on_training_set))
8
9 # compute the accuracy of the trained network on the test set
10 predict_test = model.predict(X_test)
11 accuracy_on_test_set = accuracy_score(y_test, predict_test)
12 print("Accuracy on the test set: %.2f %" % (100 * accuracy_on_test_set))
```

Questions

1. Draw a picture of the above network. Show the weight and bias values of all connections.
2. What happens if we change the value of learning rate to 0.0001, 0.01, or 1. Show the plot of the loss curve of each case and give your conclusions.
3. Show an example of overfitting.

Handwriting Digit Recognition

Training image: 3 Testing image: 6



Steps to build a neural network for handwriting digit recognition:

1. Load dataset
2. Split data for training and testing
3. Normalize the feature values
4. Build neural network model
5. Train the neural network
6. Evaluate the trained neural network

```
# Step 1. Load dataset =====
# Load the mnist dataset using keras
from tensorflow.keras.datasets import mnist

(X_train, y_train), (X_test, y_test) = mnist.load_data()

X_train = X_train.reshape(60000, 784)
X_test = X_test.reshape(10000, 784)

# Show sample images
# show one example in training set
idx = 150
x = X_train[idx, :]
print
y = y_train[idx]
plt.figure()
plt.subplot(1, 2, 1)
plt.imshow(np.reshape(x, (28, 28)), cmap=plt.cm.gray)
plt.title('Corresponding label: %i\n' % y, fontsize=20)
```

```
# Step 2. Split data for training and testing =====  
# Step 2 is not necessary since the dataset was divided into training and test set  
# show the number of training images and test images  
print("Number of training images:", len(y_train))  
print("Number of test images:", len(y_test))  
  
# Step 3. Normalize the feature values =====  
X_train = X_train.astype('float32')  
X_test = X_test.astype('float32')  
X_train /= 255  
X_test /= 255
```


Question

1. Complete step 4, 5, and 6. Show your results.
2. What happen if we do not normalize pixel values?